
가상 면접 사례로 배우는 대규모 시스템 설계 기초 2

6장. 광고 클릭 이벤트 집계

클릭 하나를 정확하게 세는 시스템 설계

무엇을 만들어야 하나

기능 요구사항

- 1 특정 광고의 최근 M분간 클릭 수 조회
- 2 최근 M분간 가장 많이 클릭된 광고 상위 100개 (매 분 갱신)
- 3 위 두 질의를 IP, user_id, 국가 기준으로 필터링

비기능 요구사항

- **정확성** — 실시간 경매(RTB)와 광고비 청구에 쓰임
- 늦게 도착하거나 중복된 이벤트 처리
- **견고성** — 일부 구성 요소가 고장 나도 동작
- 종단 간 지연은 수 분 이내

이 장의 상세 설계는 대부분 "**정확성을 어떻게 지키는가**"에 대한 이야기

규모 추정

항목	계산	결과
하루 클릭 수	DAU 10억 × 1인 1클릭	10억 건
평균 QPS	$10^9 \div 10^5$ 초 (하루 ≈ 10만 초)	약 10,000
최대 QPS	평균 × 5 가정	약 50,000
저장 공간	이벤트당 0.1KB × 10억	하루 100GB · 월 3TB

QPS(Queries Per Second): 초당 처리하는 요청 수. 여기서는 초당 들어오는 클릭 이벤트 수
광고 수는 약 200만 개, 클릭 이벤트는 매년 30%씩 증가한다고 가정

읽기보다 **쓰기가 압도적으로 많은 시스템** → 저장소 선택의 근거

질의 API는 두 개

GET /v1/ads/{ad_id}/aggregated_count

특정 광고의 기간별 클릭 수

요청 from, to, filter

응답 ad_id, count

GET /v1/ads/popular_ads

최근 M분간 클릭 수 상위 N개 광고

요청 count, window, filter

응답 ad_ids

API는 대시보드가 결과를 **읽는** 경로 · 시스템의 진짜 부담은 초당 최대 5만 건 들어오는 클릭 이벤트를 **받아 쓰는** 경로

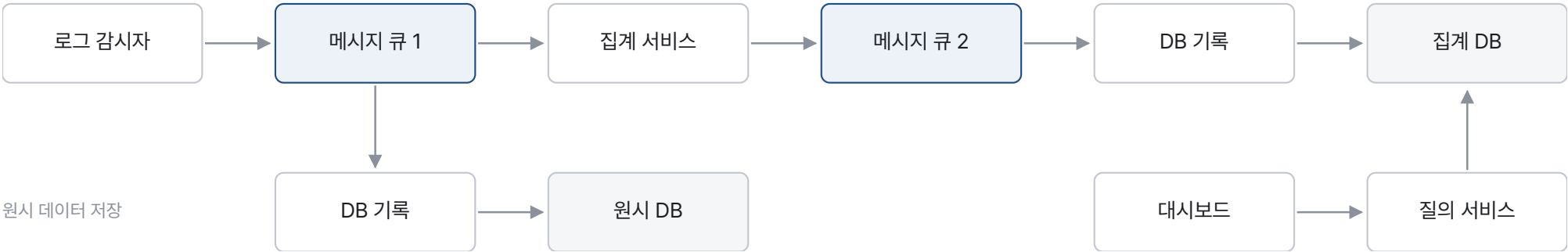
원시 데이터 vs 집계 데이터

	원시 데이터	집계 데이터
형태	클릭 이벤트 그대로 ad001 · 00:00:01 · user1 · 207.148.22.22 · USA	광고별 분 단위 클릭 수 ad001 · 202101010000 · 5
장점	원본 보존, 다시 계산 가능	용량 작음, 조회 빠름
단점	용량 큼, 조회 느림	원래 정보 손실

둘 다 저장 — 원시는 백업·재계산용(콜드), 집계는 조회용 · 쓰기 중심이라 Cassandra, InfluxDB 등

전체 구조

집계 경로



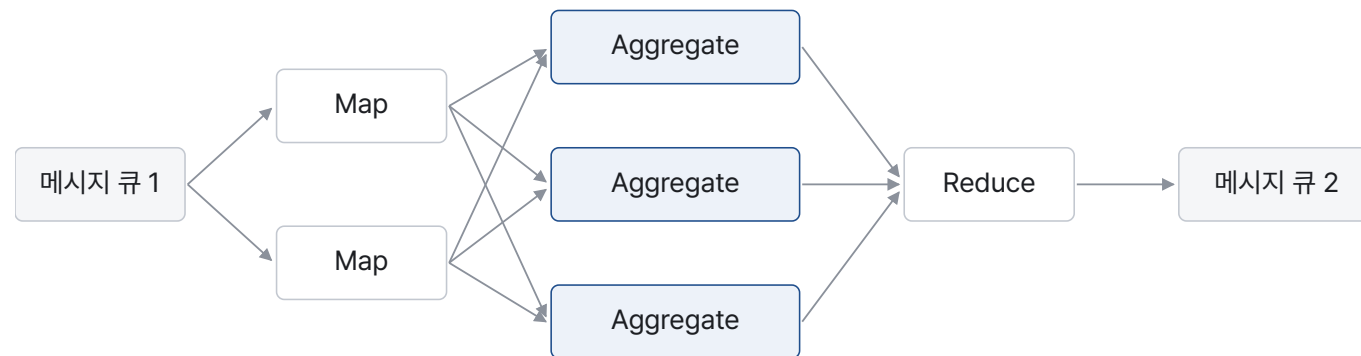
원시 데이터 저장

큐	들어가는 데이터	예시 필드
메시지 큐 1	원시 클릭 이벤트	ad_id, click_timestamp, user_id, ip, country
메시지 큐 2	분 단위 광고별 클릭 수 분 단위 상위 N개 광고	ad_id, click_minute, count update_time_minute, most_clicked_ads

메시지 큐를 **두 개** 두는 이유는 "정확히 한 번" 처리 때문 → 상세 설계에서

집계 서비스 내부: Map → Aggregate(집계) → Reduce(축약)

초당 수만 건을 한 노드가 처리할 수 없으므로, 일을 작은 단위로 나눠서 여러 노드가 동시에 처리하고 결과를 합치는 맵리듀스 구조를 DAG로 구성



Map

이벤트 정제·필터링 후 ad_id 기준으로 분배 (예: $\text{ad_id} \% 3$)

Aggregate(집계)

같은 광고의 클릭을 메모리에서 분 단위로 집계

Reduce(축약)

여러 Aggregate 노드의 결과를 하나로 합침

주요 사용 사례

클릭 이벤트 수 집계

같은 광고는 항상 같은 노드로 가므로, 노드끼리 주고받지 않고 각자 독립적으로 셀 수 있음

상위 N개 광고 반환

각 노드가 자기 몫의 상위 N만 뽑고 Reduce가 합침 → 전체 데이터를 한곳에 모으지 않아도 됨

데이터 필터링

Map 단계에서 조건별로 나눠 필터별로 미리 집계(스타 스키마) → 질의 시 다시 계산할 필요 없음

정확하게 세기 어려운 이유 세 가지

1 이 클릭은 언제 일어났나?

이벤트 시각, 늦게 온 이벤트, 워터마크, 집계 윈도우

2 한 번만 셧나?

전달 보장, 오프셋 저장, 정확히 한 번 처리

3 커지거나 고장 나도 맞나?

규모 확장, 핫스팟, 결함 내성, 모니터링과 조정

스트리밍 vs 일괄 처리

	일괄 처리 (Batch)	스트리밍 (Streaming)
입력	이미 쌓여 있는, 끝이 정해진 대량 데이터	끝없이 계속 들어오는 데이터
처리 시점	모아뒀다가 주기적으로 한꺼번에	들어오는 즉시
중요 지표	처리량	처리량 + 지연 시간
예시	MapReduce, Spark	Flink, Kafka Streams

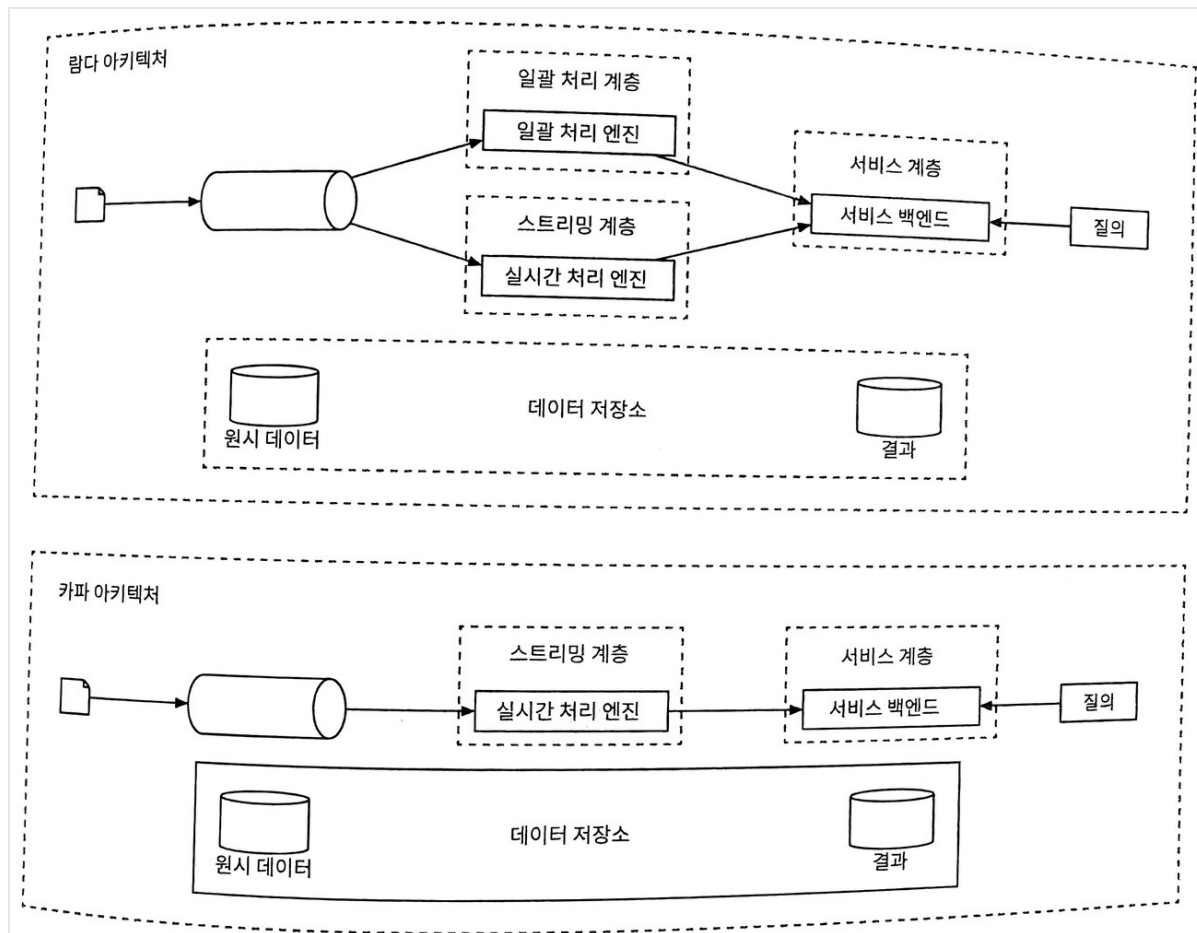
람다 아키텍처 · 둘 다 사용

- 스트리밍 경로(빠름)와 일괄 처리 경로(정확)를 따로 운영
- 같은 집계 로직을 **두 번** 유지해야 함

카파 아키텍처 · **책의 선택**

- 스트리밍 하나로 실시간 처리와 재처리를 모두 해결
- 재처리는 저장해둔 데이터를 **다시 흘려보내는** 방식

그림으로 보는 람다 vs 카파



람다

- 1 메시지 큐에서 길이 **두 갈래**로 나뉨
- 2 **일괄 처리 계층** 쌓인 데이터를 주기적으로 처리 (느리지만 정확)
- 3 **스트리밍 계층** 들어오는 즉시 처리 (빠름)
- 4 **서비스 계층** 두 결과를 모아 질의에 응답

→ 같은 집계 로직을 엔진 두 개에 따로 구현

카파

- 1 **일괄 처리 계층이 없음** 스트리밍 계층 하나뿐
- 2 **실시간 처리 엔진 하나**가 실시간 처리와 재처리를 모두 담당
- 3 **재처리** 데이터 저장소의 원시 데이터를 같은 엔진에 다시 흘려보냄

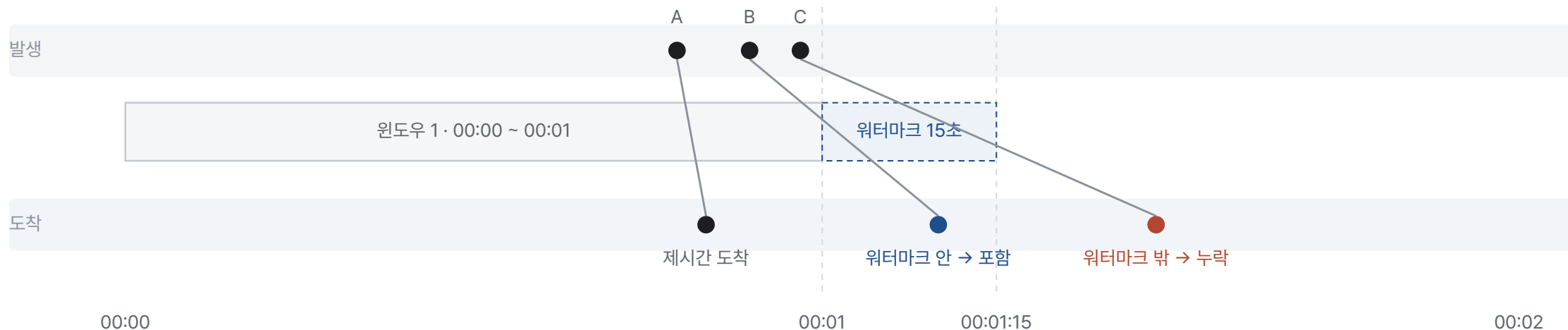
→ 코드 한 벌 · 책의 "원시 데이터로 재계산"이 바로 이 방식

이벤트 시각 vs 처리 시각

	이벤트 시각	처리 시각
기준	사용자가 실제로 클릭한 시각	집계 서버가 이벤트를 처리한 시각
장점	클릭이 일어난 구간에 정확히 집계	서버 시계라 신뢰할 수 있음
단점	클라이언트 시계에 의존, 늦게 오는 이벤트 처리 필요	늦게 도착한 이벤트가 엉뚱한 구간에 집계됨

정확성이 우선이므로 **이벤트 시각** 사용 → 남은 숙제: 늦게 오는 이벤트

워터마크: 구간을 조금 더 기다려주기



길게 잡을수록 정확도 ↑, 결과 지연 ↑ · 그래도 놓친 이벤트는 **하루 끝 조정**으로 보정

조정(reconciliation): 서로 다른 데이터를 비교해 데이터 무결성을 보증하는 작업. 하루가 끝나면 원시 데이터로 다시 집계한 결과와 실시간 집계 결과를 비교

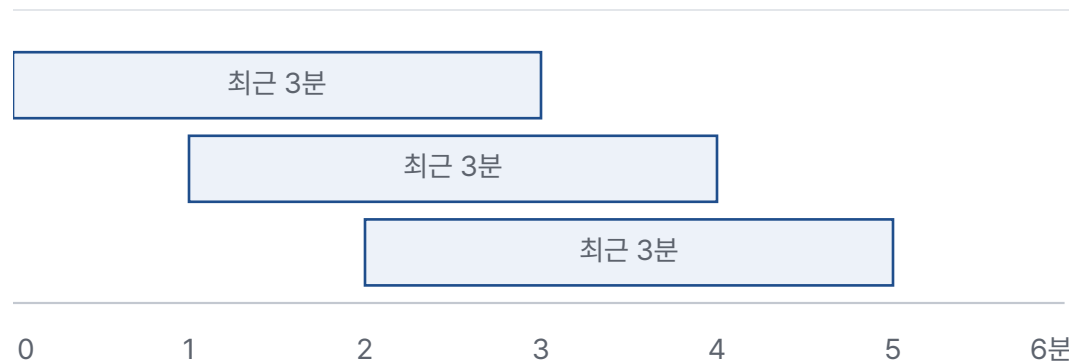
집계 윈도우: 텀블링 vs 슬라이딩

텀블링 윈도우



겹치지 않는 고정 구간
→ 분당 클릭 수

슬라이딩 윈도우



일정 길이의 구간이 1분씩 이동, 구간끼리 겹침
→ 최근 M분 상위 100개

전달 보장: 이벤트는 몇 번 처리되는가

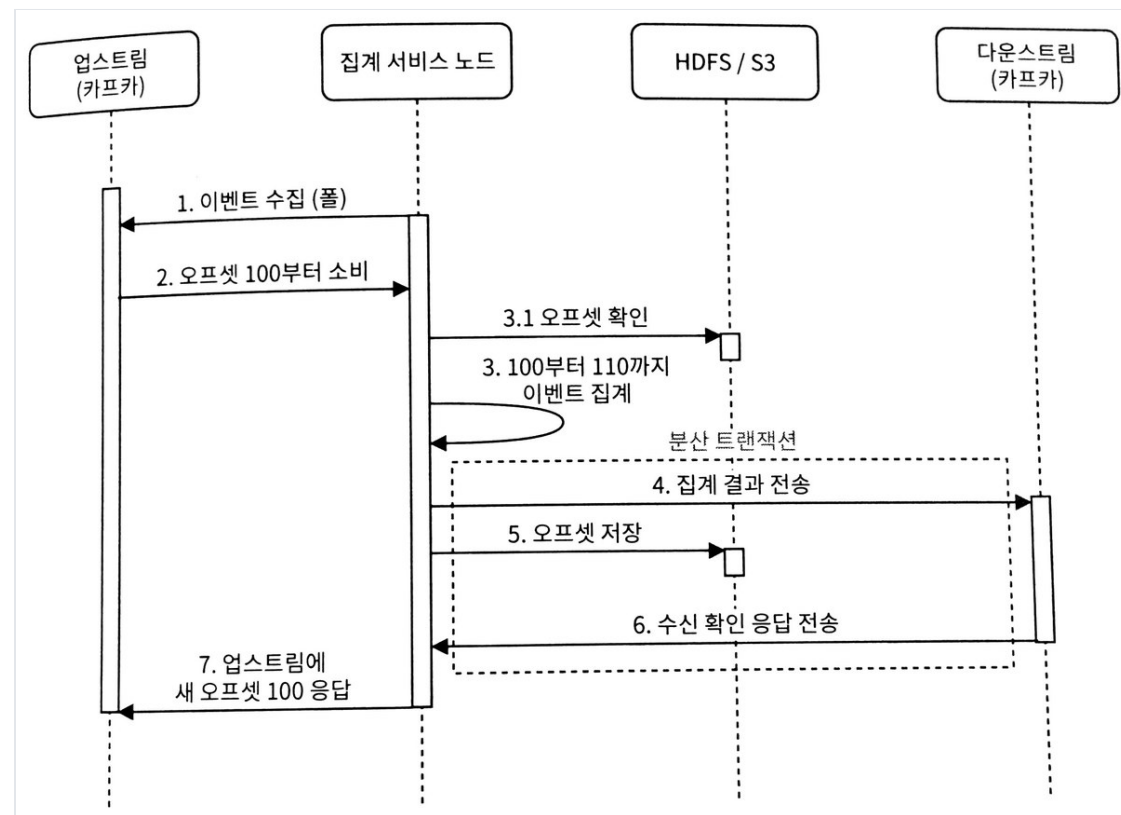
보장 수준	의미	위험
최대 한 번 at-most-once	실패해도 다시 보내지 않음	유실 가능
최소 한 번 at-least-once	성공이 확인될 때까지 다시 보냄	중복 가능
정확히 한 번 exactly-once	유실도 중복도 없음	구현이 복잡

중복이 생기는 원인 · 클라이언트가 같은 이벤트를 다시 보내는 경우, 집계 노드가 처리 도중 죽었다가 다시 읽는 경우

광고 클릭은 과금 데이터라 작은 비율의 오차도 큰 금액 차이 → **정확히 한 번**이 필요

정확히 한 번까지: 오프셋 저장 방식의 발전

- 1 오프셋 기록 없음** 중복 · 유실
노드가 죽으면 어디까지 처리했는지 모름. 처음부터 읽으면 중복, 최신부터 읽으면 유실
- 2 오프셋 기록** 유실
HDFS·S3 같은 외부 저장소에 오프셋 저장 → 재시작 위치는 알. 하지만 결과를 보내기 전에 오프셋부터 저장하고 죽으면 그 사이 이벤트가 사라짐
- 3 수신 확인 후 오프셋 저장** 중복
하류가 결과를 받았다는 확인(ack)을 받은 뒤 저장 → 유실은 없음. 하지만 확인과 저장 사이에 죽으면 다시 처리해서 중복 (최소 한 번)
- 4 분산 트랜잭션** 정확히 한 번
결과 전송과 오프셋 저장을 하나의 트랜잭션으로 묶어 둘 다 성공하거나 둘 다 취소



구성 요소별 규모 확장

메시지 큐

- ad_id를 파티션 키로 → 같은 광고는 같은 파티션
- 파티션 수는 **처음부터 넉넉히**
- 소비자 추가 시 리밸런싱은 한가한 시간에
- 필요하면 지역·사업 유형별로 토픽 분리

집계 서비스

- 노드 안에서 멀티스레드로 광고별 처리
- YARN 같은 자원 관리자 위에 배포해 노드 추가

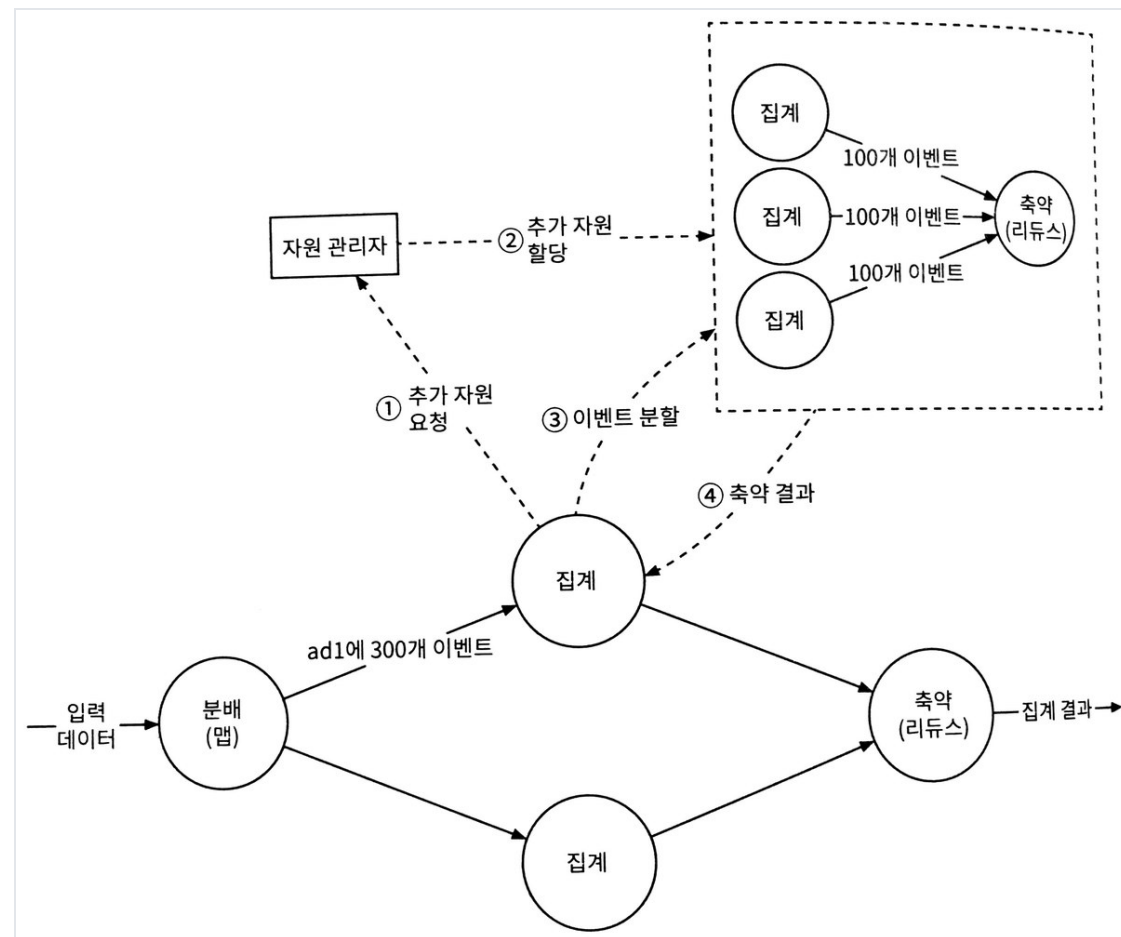
데이터베이스

- Cassandra는 기본적으로 수평 확장 지원
- 안정 해시 + 가상 노드로 노드 추가 시 데이터 재배치 최소화

파티션을 나중에 늘리면 같은 광고의 이벤트가 **다른 파티션으로** 갈 수 있음

핫스팟

- 인기 광고에 클릭이 몰리면 그 광고를 맡은 집계 노드만 과부하
- ① 해당 노드가 자원 관리자에게 **추가 자원 요청**
- ② 자원 관리자가 집계 노드를 추가로 할당
- ③ 이벤트를 나눠 각 노드가 집계 (예: 300개 → 100개씩)
- ④ 결과를 축약해 원래 노드로 돌려줌



결함 내성

문제

- 집계 상태는 메모리에 있어 노드가 죽으면 사라짐
- 업스트림(Kafka) 처음부터 다시 읽으면 너무 느림

해결: 스냅샷

- 오프셋과 집계 상태(예: 최근 상위 N개)를 주기적으로 **스냅샷**
- 노드가 죽으면 새 노드가 마지막 스냅샷을 불러옴
- 스냅샷 이후 이벤트만 업스트림에서 다시 처리

모니터링과 조정(reconciliation)

지속적 모니터링

- **지연 시간** — 각 단계마다 지연이 더해질 수 있으므로, 중요한 지점마다 타임스탬프를 남겨 추적할 수 있어야 함
- **메시지 큐 크기** — 큐에 쌓이는 양이 계속 늘어나면 집계 서비스 노드를 더 추가해야 할 수 있음
- **시스템 자원** — 집계 노드의 CPU, 디스크, JVM 등

하루 끝 조정

- 배치 작업으로 원시 데이터를 이벤트 시각 기준으로 다시 집계
- 실시간 집계 결과와 비교
- 워터마크 이후 도착분 때문에 **완전히 일치하지 않을 수 있음**

정확성을 지키는 장치들

문제	장치	대가
늦게 도착한 이벤트	워터마크 + 하루 끝 조정	결과 지연
중복 집계·유실	결과 전송과 오프셋 저장을 원자적으로	분산 트랜잭션 복잡도
노드 장애	스냅샷 + 이후 이벤트 재처리	스냅샷 저장 비용
집계 로직 버그	원시 데이터로 재계산	원시 데이터 저장 공간
트래픽 증가·몰림	파티셔닝, 노드 추가	운영 복잡도

궁금했던 점

재계산 서비스는 왜 따로 둘까?

집계 서비스와 코드는 같은데, 입력만 원시 데이터로 바뀌서 별도로 운영하는 이유

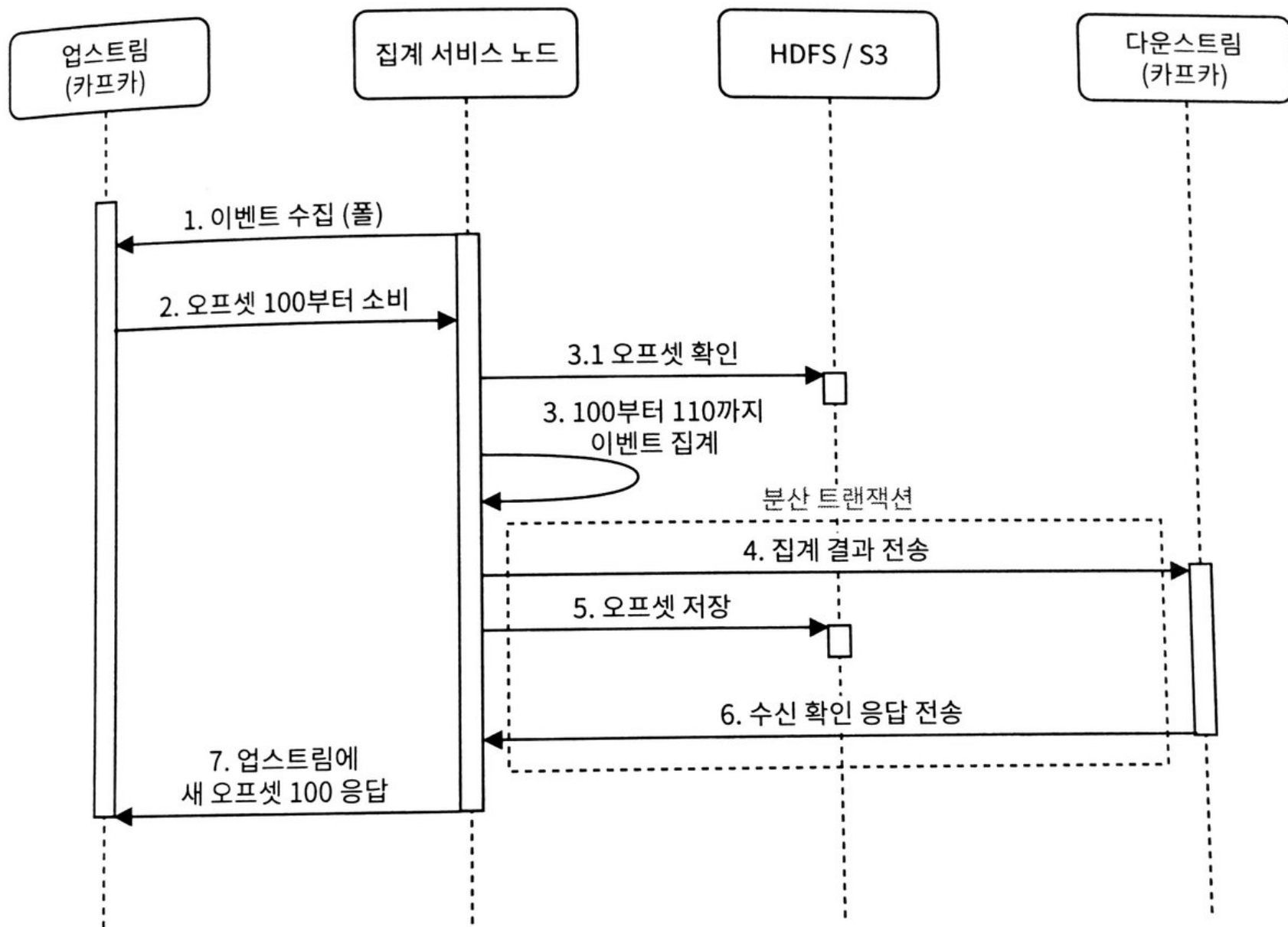
같이 돌리면 생기는 문제

- **실시간 처리가 느려짐** — 몇 달 치 데이터를 재처리하면 자원을 뺏겨 "수 분 이내" 요구사항이 깨짐
- **시간 처리가 꼬임** — 워터마크는 이미 지나갔으므로 과거 이벤트는 전부 "아주 늦게 온 이벤트"로 버려짐
- **오프셋·상태가 섞임** — 정확히 한 번을 위한 기록이 섞이면 장애 복구 불가
- **고친 코드를 검증할 수 없음** — 재계산은 대개 버그 수정 때문

책의 재처리 방법

- 1 원시 데이터 저장소에서 **문제가 된 기간의 데이터**를 읽음
- 2 실시간 집계 서비스와 분리된 **전용 집계 서비스**로 보내 다시 집계
- 3 결과는 메시지 큐 2를 거쳐 **집계 DB의 해당 구간을 고쳐 씀**
- 4 재계산이 진행되는 동안에도 **실시간 처리는 기존 집계 서비스가 계속 맡음**

코드는 같지만 **입력과 실행을 분리** → 실시간 처리를 지키면서 과거 결과를 고칠 수 있음



집계 결과를 왜 DB에 바로 쓰지 않을까?

답: "임시로 써두기"가 되는 곳이 필요해서

DB(Cassandra)에 바로 쓰면

쓰는 순간 바로 저장되고 바로 보임 → 임시 상태가 없음

오프셋 저장이 실패해도 이미 들어간 결과를 되돌릴 수 없음 → 재시작 시

중복

Kafka(메시지 큐 2)로 보내면

트랜잭션 안에서 쓴 메시지는 "미확정" 상태로 대기

읽는 쪽(DB 기록)은 확정된 메시지만 읽음

확정 지시가 오면 그때 보이고, 취소되면 버려짐 → **2단계 커밋에 참여 가능**

남은 구간(큐 2 → DB)은 (ad_id, click_minute) 키로 덮어쓰는 **역등한 쓰기**로 보완 → 같은 결과가 두 번 와도 숫자가 그대로

분산 트랜잭션은 어떻게? 2단계 커밋

코디네이터(조율자) = 참여자들에게 묻고 최종 결정을 내림 → 집계 노드

참여자 = 실제로 저장하는 곳 → Kafka(결과), HDFS/S3(오프셋)

1

준비

코디네이터가 Kafka와 HDFS/S3에 "확정할 수 있어?"라고 묻고, 각자 쓰기를 임시로 디스크에 저장해둔 뒤 응답

"가능" = 내 몫을 임시로 안전하게 저장해뒀고, 확정하라면 반드시 확정할 수 있음

2

판단

두 곳 모두 "가능"이면 확정

서버·네트워크 장애, 디스크 가득 참, 타임아웃 등으로 하나라도 **"불가능"**이거나 **무응답**이면 취소

3

확정 또는 취소

결정을 전달 → 결과와 오프셋이 함께 반영되거나 함께 버려짐

대가: 왕복 통신이 늘고, 코디네이터가 멈추면 참여자가 결정을 기다리며 묶임